



UNIVERSIDADE FEDERAL DA PARAÍBA
CAMPUS IV - RIO TINTO
SISTEMAS DE INFORMAÇÃO

Equipe: Rhauany Aragão, Jamilly Santos, Maria Rafaela, Mariano Santos.

Disciplina: Análise e Projeto de Sistemas

Professor: Dorgival Pereira da Silva Netto

Tema: Sistema de gestão de granja

Agro Plus

DOCUMENTO DE MODELAGEM DE CLASSES – AGROPLUS

1. ARQUITETURA UTILIZADA: MVC (MODEL-VIEW-CONTROLLER)

O sistema AgroPlus será desenvolvido seguindo o padrão arquitetural MVC, que separa a aplicação em três camadas principais:

- Model (Modelo): Responsável pelos dados e regras de negócio. Contém as classes que representam as entidades do sistema.
 - View (Visão): Responsável pela interface com o usuário. Apresenta os dados e captura as entradas do usuário.
 - Controller (Controlador): Intermedia as requisições entre View e Model, processando eventos, validando permissões e direcionando ações.
-

2. DESCRIÇÃO DAS CLASSES (CAMADA MODEL)

2.1. Classe Funcionario (Unificada)

Responsabilidade: Representa todos os trabalhadores da granja, incluindo o patrão (administrador) e demais funcionários. O patrão possui permissão especial para acessar todas as funcionalidades do sistema.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do funcionário

nomeCompleto	string	Nome completo do funcionário
cpf	string	CPF do funcionário (único)
cargo	string	Cargo do funcionário (ex: Gerente, Auxiliar, Veterinário, Patrão)
telefone	string	Número de telefone para contato
email	string	E-mail do funcionário (usado para login)
senha	string	Senha criptografada para acesso ao sistema
dataAdmissao	Date	Data de contratação
tipoAcesso	string	Nível de acesso: "administrador" (patrão) ou "comum"

Métodos:

Método	Descrição
autenticar(email, senha)	Verifica credenciais e retorna true se válido
cadastrar(dados)	Cadastra um novo funcionário (apenas administradores)
alterarCargo(novoCargo)	Altera o cargo do funcionário (apenas administradores)

demitir()	Remove o funcionário do sistema (apenas administradores)
getPermissoes()	Retorna lista de permissões baseadas no tipoAcesso
alterarSenha(novaSenha)	Permite ao funcionário alterar sua própria senha

Regras de Negócio:

- O patrão (tipoAcesso = "administrador") pode acessar todas as funcionalidades do sistema.
- Apenas patrões podem excluir registros, demitir funcionários e alterar cargos.
- O e-mail deve ser único no sistema.

2.2. Classe Animal

Responsabilidade: Representa os animais da granja, armazenando informações individuais de cada um.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do animal
identificacaoUnica	string	Código único do animal (ex: GRN-001)
especie	string	Espécie do animal (ex: Frango, Galinha)

idade	int	Idade em meses
pesoAtual	float	Peso atual do animal em kg
situacaoSaude	string	Saudável, Doente, EmTratamento ou Morto
dataNascimento	Date	Data de nascimento do animal
lote	string	Identificação do lote ao qual pertence
ativo	boolean	Indica se o animal está ativo na granja

Métodos:

Método	Descrição
cadastrar(dados)	Cadastra um novo animal no sistema
atualizarPeso(novoPeso)	Atualiza o peso atual do animal
atualizarSaude(novaSituacao)	Atualiza a situação de saúde do animal
getHistoricoCompleto()	Retorna o histórico completo do animal
excluir()	Exclui o animal (apenas administradores)

Regras de Negócio:

- Cada animal deve possuir identificação única.
- Apenas administradores (patrão) podem excluir registros de animais.
- Quando um animal é excluído, seus registros associados (peso, aplicações) devem ser mantidos para histórico.

2.3. Classe RegistroPeso

Responsabilidade: Armazena o histórico de pesagens de cada animal.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do registro
animalId	int	ID do animal pesado
peso	float	Peso registrado em kg
dataPesagem	Date	Data da pesagem

Métodos:

Método	Descrição
registrar(animalId, peso)	Registra uma nova pesagem para o animal
listarPorAnimal(animalId)	Lista todos os registros de peso de um animal

2.4. Classe VacinaMedicamento

Responsabilidade: Representa os produtos (vacinas e medicamentos) utilizados na granja.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do produto
nome	string	Nome do produto
tipo	string	"Vacina" ou "Medicamento"
lote	string	Número do lote do produto
dataValidade	Date	Data de validade do produto
quantidadeEstoque	int	Quantidade disponível em estoque

Métodos:

Método	Descrição
aplicar(animalId, dose, dataAplicacao)	Aplica o produto em um animal
atualizarEstoque(quantidade)	Atualiza a quantidade em estoque

2.5. Classe AplicacaoProduto

Responsabilidade: Registra a aplicação de vacinas/medicamentos nos animais.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único da aplicação

animalId	int	ID do animal que recebeu a aplicação
produtoId	int	ID do produto aplicado
dose	string	Dosagem aplicada
dataAplicacao	Date	Data da aplicação
funcionarioResponsavel	string	Nome do funcionário que aplicou
observacoes	string	Observações adicionais

Métodos:

Método	Descrição
registrarAplicacao(dados)	Registra uma nova aplicação no sistema

2.6. Classe VisitaVeterinaria

Responsabilidade: Registra visitas veterinárias e atendimentos realizados.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único da visita
dataVisita	Date	Data da visita

funcionarioId	int	ID do funcionário (veterinário) que realizou
objetivo	string	Objetivo da visita
observacoes	string	Observações sobre a visita
animaisAtendidos	List[int]	Lista de IDs dos animais atendidos
Métodos:		
Método	Descrição	
registrarVisita(dados)	Registra uma nova visita veterinária	
listarVisitasPorPeriodo(dataIni, dataFim)	Lista visitas em um período específico	

2.7. Classe ProducaoDiaria

Responsabilidade: Registra a produção diária da granja.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do registro
data	Date	Data da produção (obrigatória)
quantidadeOvos	int	Quantidade produzida
lote	string	Lote referente à produção

observacoes	string	Observações adicionais
-------------	--------	------------------------

Métodos:

Método	Descrição
registrarProducao(dados)	Registra a produção de um dia
getProducaoPorPeriodo(dataIni, dataFim)	Obtém produção em um período
getRelatorioProducao()	Gera relatório de produção

Regras de Negócio:

- O campo data é obrigatório e não pode ser nulo.

2.8. Classe Estoque

Responsabilidade: Controla o estoque de produtos da granja.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do registro
produtoId	int	ID do produto em estoque
nomeProduto	string	Nome do produto
quantidadeAtual	float	Quantidade atual em estoque

unidadeMedida	string	Unidade de medida (kg, L, unidade)
estoqueMinimo	float	Quantidade mínima permitida
estoqueMaximo	float	Quantidade máxima permitida
localArmazenamento	string	Local onde o produto é armazenado

Métodos:

Método	Descrição
registrarEntrada(quantidade, notaFiscal)	Registra entrada de produtos no estoque
registrarSaida(quantidade, motivo)	Registra saída de produtos do estoque
verificarEstoqueBaixo()	Verifica se o estoque está abaixo do mínimo
gerarAlerta()	Gera alerta de estoque baixo

Regras de Negócio:

- Estoque não pode ficar negativo.
- O método registrarSaida() deve validar se há quantidade suficiente.

2.9. Classe Alimento

Responsabilidade: Representa os alimentos utilizados na alimentação dos animais.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do alimento
nome	string	Nome do alimento
tipo	string	"Ração", "Suplemento" ou "Mineral"
fornecedor	string	Nome do fornecedor
validade	Date	Data de validade do alimento
precoUnitario	float	Preço por unidade

Métodos:

Método	Descrição
consumir(animalId, quantidade)	Registra o consumo de alimento por um animal
atualizarPreco(novoPreco)	Atualiza o preço do alimento

2.10. Classe Relatorio

Responsabilidade: Gera e gerencia relatórios do sistema.

Atributos:

Atributo	Tipo	Descrição
id	int	Identificador único do relatório

tipo	string	"Producao", "Estoque", "Saude", "Desempenho"
dataGeracao	Date	Data de geração do relatório
parametros	Map	Parâmetros utilizados na geração
conteudo	blob	Conteúdo do relatório

Métodos:

Método	Descrição
gerar(tipo, parametros)	Gera um novo relatório
exportarPDF()	Exporta o relatório em formato PDF
exportarExcel()	Exporta o relatório em formato Excel

2.11. Classe HistoricoAnimal (Agregação)

Responsabilidade: Agrega todo o histórico de um animal (pesos, aplicações, produção associada).

Atributos:

Atributo	Tipo	Descrição
animal	Animal	Dados básicos do animal
listaPesos	List	Lista de registros de peso
listaAplicacoes	List	Lista de aplicações de produtos

listaProducaoAssociada	List	Produção associada ao animal
------------------------	------	------------------------------

Métodos:

Método	Descrição
carregarHistorico(animalId)	Carrega todo o histórico do animal
exibirCompleto()	Exibe o histórico completo formatado

3. RELACIONAMENTOS ENTRE CLASSES

Relacionamento	Descrição	Cardinalidade
Funcionario → Animal	Um funcionário cadastra/gerencia vários animais	1 → 0..*
Funcionario → VisitaVeterinaria	Um funcionário pode realizar várias visitas	1 → 0..*
Funcionario → AplicacaoProduto	Um funcionário pode realizar várias aplicações	1 → 0..*
Animal → RegistroPeso	Um animal tem vários registros de peso	1 → 0..*
Animal → AplicacaoProduto	Um animal recebe várias aplicações	1 → 0..*

Animal → ProducaoDiaria	Um animal pode estar associado a várias produções	1 → 0..*
VacinaMedicamento → AplicacaoProduto	Um produto pode ser usado em várias aplicações	1 → 0..*
VisitaVeterinaria → Animal	Uma visita pode atender vários animais	1 → 0..*
Estoque → Alimento	Um estoque armazena vários alimentos	1 → 0..*

4. REGRAS DE NEGÓCIO CONSOLIDADAS

1. Identificação única de animais: Cada animal deve possuir um código identificador único (ex: GRN-001).
2. Exclusão restrita: Somente administradores (patrão) podem excluir registros de animais e funcionários.
3. Estoque não negativo: O sistema não permite que o estoque fique com quantidade negativa.
4. Data obrigatória: Todo registro de produção diária deve ter uma data associada.
5. Acesso do patrão: O patrão (administrador) tem permissão para acessar todas as funcionalidades do sistema.
6. Login único: Cada funcionário possui um e-mail único para acesso ao sistema.

5. CAMADAS CONTROLLER (RESUMO)

Controller

Responsabilidades

AuthController	Login, logout, sessão, verificação de permissões (incluindo permissão especial para patrão)
AnimalController	CRUD de animais, atualização de peso/saúde, exclusão (restrita)
EstoqueController	Controle de entradas/saídas, alertas de estoque baixo
ProducaoController	Registro e consulta de produção diária
RelatorioController	Geração e exportação de relatórios
FuncionarioController	Cadastro, alteração de cargo, demissão (apenas patrão)

6. CAMADAS VIEW (RESUMO)

View	Funcionalidades
LoginView	Tela de autenticação
DashboardView	Painel principal com resumos, gráficos e alertas
AnimalView	Cadastro, consulta, histórico, peso, saúde dos animais
EstoqueView	Controle de estoque, entradas/saídas, alertas
ProducaoView	Registro e consulta de produção diária

RelatorioView

Geração e visualização de relatórios

FuncionarioView

Gerenciamento de funcionários (apenas patrão)

Elaborado por: Rhauany Aragão, Jamilly Santos, Maria Rafaela, Mariano Santos

Disciplina: Análise e Projeto de Sistemas

Data: 19/06/2026